Московский Авиационный Институт
(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу «Операционные системы»

Группа: М8О-209БВ-25
Студент: Алексеев Сергей Олегович
Преподаватель: Бахарев В.Д.
Оценка: _____
Дата: 28.09.26

Москва, 2026

Постановка задачи

Вариант 1.

Родительский процесс создаёт дочерний процесс. Имя файла, в который дочерний процесс должен записывать результаты работы, вводится пользователем в родительском процессе и передаётся дочернему при его запуске. Родительский и дочерний процессы реализованы отдельными программами.

Пользователь вводит команды в формате «число число число<конец строки>». Родительский процесс передаёт введённые числа дочернему процессу через pipe, подключённый к стандартному вводу дочернего процесса. Дочерний процесс вычисляет сумму чисел каждой строки и записывает полученный результат в указанный файл. Тип чисел — int, количество чисел в строке произвольное.

Общий метод и алгоритм решения

Программа состоит из двух исполняемых файлов: parent и child. Сначала parent считывает имя выходного файла и создаёт неименованный канал pipe. Затем с помощью fork() создаётся дочерний процесс.

В дочерней ветви закрывается дескриптор pipe, предназначенный для записи. Системный вызов dup2() подключает читающий конец pipe к стандартному вводу STDIN_FILENO. После этого execl() заменяет код дочернего процесса программой child, передавая ей имя выходного файла через аргументы командной строки.

Родительская ветвь закрывает читающий конец pipe, считывает введённые пользователем данные системным вызовом read() и передаёт их дочернему процессу через write(). После получения EOF родитель закрывает pipe и ожидает завершения дочернего процесса с помощью waitpid().

Программа child открывает выходной файл системным вызовом open() и читает данные из стандартного ввода. Числа разбираются посимвольно, поэтому корректность обработки не зависит от того, на какие части поток данных был разделён отдельными вызовами read(). При получении символа конца строки к сумме добавляется последнее число, сумма преобразуется в последовательность символов и записывается в файл.

Использованные системные вызовы:

- pipe() — создаёт неименованный канал для передачи данных между процессами.
- fork() — создаёт дочерний процесс.

• dup2() — переназначает стандартный ввод дочернего процесса на читающий конец pipe.
• execl() — заменяет программу дочернего процесса программой child.
• read() — считывает имя файла и данные из стандартного ввода/pipe.
• write() — передаёт данные через pipe и записывает результат в выходной файл.
• open() — открывает или создаёт выходной файл.
• close() — закрывает файловые дескрипторы.
• waitpid() — ожидает завершения созданного дочернего процесса.

Код программы

parent.c

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>


int main(){
    char filename[100];


    ssize_t filename_length = read(STDIN_FILENO, filename, sizeof(filename) - 1);


    if (filename_length <= 0) {
        return 1;
    }


    if (filename[filename_length - 1] == '\n') {
        filename[filename_length - 1] = '\0';
    } else {
        filename[filename_length] = '\0';
    }


    int pipefd[2];


    if (pipe(pipefd) == -1){
        return 1;
    }
    pid_t pid = fork();
    if (pid < 0){
        close(pipefd[0]);
        close(pipefd[1]);
        return 1;
    }
    else if (pid == 0){
        close(pipefd[1]);


        if (dup2(pipefd[0], STDIN_FILENO) == -1) {
            close(pipefd[0]);
            return 1;
        }
        close(pipefd[0]);
        execl("./child", "child", filename, NULL);
        return 1;
    }


    else{
        close(pipefd[0]);


        char buffer[100];


        while (1) {
            ssize_t bytes_read = read(STDIN_FILENO, buffer, sizeof(buffer));


            if (bytes_read == 0) {
                break;
            }


            if (bytes_read < 0) {
                close(pipefd[1]);
                waitpid(pid, NULL, 0);
```

```
                return 1;
            }

            ssize_t bytes_written = write(pipefd[1], buffer, bytes_read);

            if (bytes_written != bytes_read) {
                close(pipefd[1]);
                waitpid(pid, NULL, 0);
                return 1;
            }
        }

        if (close(pipefd[1]) == -1) {
            waitpid(pid, NULL, 0);
            return 1;
        }

        if (waitpid(pid, NULL, 0) == -1) {
            return 1;
        }
    }

    return 0;
}
```

child.c

```
#include <unistd.h>
#include <sys/types.h>
#include <fcntl.h>


void calculate_sum(char symbol, int *sum, int *number, int *sign) {
    if (symbol >= '0' && symbol <= '9') {
        *number = *number * 10 + (symbol - '0');
    }


    else if (symbol == '-') {
        *sign = -1;
    }


    else if (symbol == ' ') {
        *sum = *sum + (*number * *sign);
        *number = 0;
        *sign = 1;
    }
}


int int_to_string(int sum, char *result) {
    int length = 0;
    int is_negative = 0;


    if (sum < 0) {
        is_negative = 1;
        sum = -sum;
    }


    if (sum == 0) {
        result[0] = '0';
        length = 1;
    }


    while (sum > 0) {
        int digit = sum % 10;
        result[length] = digit + '0';
        length++;
        sum = sum / 10;
    }
    for (int i = 0; i < length / 2; i++) {
        char temp = result[i];
        result[i] = result[length - i - 1];
        result[length - i - 1] = temp;
    }


    if (is_negative) {
        for (int i = length; i > 0; i--) {
            result[i] = result[i - 1];
        }


        result[0] = '-';
        length++;
    }


    result[length] = '\n';
    length++;


    return length;
}
```

```
int main(int argc, char *argv[]) {
    if (argc < 2) {
        return 1;
    }
    int file_fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (file_fd < 0) {
        return 1;
    }

    char buffer[100];

    int sum = 0;
    int number = 0;
    int sign = 1;

    while (1) {
        ssize_t bytes_read = read(STDIN_FILENO, buffer, sizeof(buffer));
        if (bytes_read == 0) {
            break;
        }

        if (bytes_read < 0) {
            close(file_fd);
            return 1;
        }

        for (ssize_t i = 0; i < bytes_read; i++) {
            if (buffer[i] == '\n') {
                sum = sum + (number * sign);
                number = 0;
                sign = 1;

                char result[20];
                int result_length = int_to_string(sum, result);

                if (write(file_fd, result, result_length) == -1) {
                    close(file_fd);
                    return 1;
                }
                sum = 0;
            }
            else {
                calculate_sum(buffer[i], &sum, &number, &sign);
            }
        }
    }

    if (close(file_fd) == -1) {
        return 1;
    }

    return 0;
}
```

Протокол работы программы

Перед запуском обе программы были скомпилированы с включёнными предупреждениями компилятора:

```
gcc -Wall -Wextra -Werror child.c -o child
gcc -Wall -Wextra -Werror parent.c -o parent
```

При тестировании в качестве выходного файла использовался result.txt. После имени файла были введены несколько строк с положительными, отрицательными и нулевыми значениями. Завершение ввода выполнялось с помощью EOF.

```
$ ./parent
result.txt
10 20 30
-10 20 -5
100 -200 50
0 0 0
1 -2 3 -4 5
^D
$ cat result.txt
60
5
-50
0
3
```

Полученные значения соответствуют ожидаемым суммам для каждой введённой строки.

Вывод

В ходе лабораторной работы были изучены создание дочернего процесса и организация межпроцессного обмена данными с помощью неименованного канала pipe. Была реализована передача пользовательского ввода из родительского процесса дочернему с перенаправлением стандартного ввода через dup2() и запуском отдельной программы с помощью execl(). Основной сложностью стала корректная обработка потока данных независимо от границ отдельных вызовов read(), поэтому числа обрабатываются посимвольно с сохранением текущего состояния. В результате программа корректно вычисляет суммы введённых целых чисел и записывает их в указанный файл.